International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

IMPLEMENTATION OF MODIFIED LOW-POWER 8×8 SIGNED DADDA MULTIPLIER

Manash Chanda
Lecturer, Meghnad Saha Institute of Technology
manash.bst@gmail.com

Sankalp Jain
Student, Meghnad Saha Institute of Technology
sankalp.0001@gmail.com

Dr. Anup Dandapat
Faculty, Jadavpur University
anup.dandapat@gmail.com

Abstract

Multiplication algorithms have considerable effect on processors performance. A new high speed, low-power multiplication algorithm has been presented using modified Dadda tree structure. Two important modifications have been implemented in partial product generation and reduction stage. In this paper, we proposed a new algorithm to reduce power, delay, and transistor count of a multiplication algorithm implemented using Two's complement negation rules. This work presents a novel design for Dadda multiplication algorithms. The proposed multiplication algorithm has achieved 44 per cent improvement in transistor count, 90 per cent reduction in delay and 89.9 per cent improvement in power consumption in compared with conventional designs.The modified version greatly reduces power required and number of adders.

Keywords

Low-Power, Dadda multiplier, signed multiplier, 2's complement and high speed

I. INTRODUCTION

In computers one of the most basic operations is addition and multiplication. Repeated additions are by use of multiplication in which multiplicand is added repeatedly till multiplicand times. In computers the negative numbers are shown by signed numbers. In digital logic multiplication of negative weight is not possible therefore there is need for different methods.

The multipliers involve three distinct phases. First, the matrix of partial products is formed. This forms an N-row skewed matrix of partial product terms where each row contains N single bit terms. In the second phase, the partial product matrix is reduced with carry save adders to a height of two terms. Finally, in the third phase, the two terms are added with a carry propagating adder to generate the product. It is usually assumed that a carry lookahead adder is used for the addition.

This paper is concerned with the first and second phases of the signed multiplication. In reduced sign extension algorithm presented by Luigi Dadda, described in [1], the sign bits are extended so that the multiplication does not contain any negative weight bit. In this way in 8×8 bit multiplier 16 columns has to be added instead of 8 in unsigned multiplier.

Throughout this paper, both input operands are assumed to be n-bit signed words. Section 2 reviews the two main previous multiplier reduction approaches. Section 3 presents the modified Dadda approach to multiplier reduction in signed multiplication without using sign extension method. Section 4 presents the results for multiplier of size 8 bits. Finally, Section 5 summarizes the conclusions.

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

II. PREVIOUS WORK

A. Dadda Approach

In Luigi Dadda's [1] approach the unsigned multiplier was used in reduced sign extension so that the partial products are held in input registers whose content is constant from n +1 to 2n clock periods. A two's complement number A can be extended to any length by repeating the sign bit ***a*** as many times as needed:

$a_3a_2a_1a_0. = a_3a_3a_2a_1a_0. = a_3a_3a_3a_2a_1a_0.$

The leftmost *a* has a negative weight (-2^{n-2+h}) if *h* sign bits are used, the remaining *a* (if any) has positive weight corresponding to their position. Both the multiplier and the multiplicand are sign extended to 2n bits, then multiplied. The only part of the array (matrix of partial products) of interest is the one composed by the *2n* rightmost columns. The only ones affecting the desired product (not-sign-extended) and composed exclusively by positive weight terms.
This algorithm is the "reduced sign-extended array" which will consequently give the product of factors in two's complement form. In the paper it was showed that the unsigned numbers multiplier can be extended to signed multiplication by use of a input registers whose values are constant in n+1 to 2n clocks period, makes too many redundant calculations. It extends the two's complement number so that there is no negative multiplication.

The purpose of this writing is to show that the unsigned Dadda multiplier, which is more clearly described in [2] can be used for signed multiplication without signed extension of Two's complement numbers by changing some of the partial products which are generated. In Dadda's reduced sign extension algorithm 2n columns of addition has to be done with n-bit multiplier which requires more power than the unsigned multiplier.

B. Crawley and Amratunga Approach

In this paper *Crawley and Amratunga* [2] they have implemented 8 bit Dadda multiplier in a 3μm CMOS using an automated layout package. Instead of the carry save adders which proceeds in a regular but a slower manner, registers are used. The technique of pipelining increases the utilization of the processing elements at the expense of the cost incurred by adding the required register stages. For long streams of data, as may be encountered in signal processing or certain types of computer arithmetic, pipelining provides a simple means of achieving a highly advantageous increase in the throughput of the system.The final carry look-ahead adder is also pipelined which is the main speed limit. By pipelined approach the numbers of stages are reduced at the expense of slightly less regular layout.

The design would be very time consuming to layout by hand because of the irregular architecture. The time delay of the design proposed is less but the power consumed increases due to usage of large number of registers.

C. Asadee Approach

This work [4] presents a design for Dadda multiplication algorithms. The proposed multiplication algorithm includes a carry hybrid adder which has less transistor count and less delay. In this paper, a 1.3V, 64-bit carry hybrid adder is presented for fast, low voltage applications. The proposed Smith multiplication algorithm has a recognition component, a radix 4-smith encoder, a 16bit Dadda tree, a 4bit counter tree and a final carry hybrid adder. The recognition component selects between Dadda tree and the counter tree as per the precision requirement. The smith encoder and the inner product generator produce partial products and the unused circuits are able to continue in their static states. Smith multiplication algorithm is implemented with five pipeline steps, with an input enable signal being used as a power reduction part for each step. The proposed modified Dadda tree architecture divides n×n Dadda multiplication algorithm into 2n parts. The AND gates are used recursively with high speed switches. The Half adders and Full adders are used recursively such that the

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

tree is more structured. The 64-bit carry hybrid adder has parallel structure.

As shown clearly in [5], the final adder for a Dadda multiplier would require a larger CLA adder than the conventional Wallace multiplier. If a CLA is used, the increased length may require an additional level of look ahead that would cause the Dadda multiplier to be slightly slower.

D. Waters and Swartzlander Approach

In their paper Waters and Swartzlander [6] present a modification of Wallace multiplier [7]. It minimizes the number of half adders used in the Wallace multiplier as a half adder does not decrease the number of partial products. In initial partial product array of the first phase the bits are shifted left to form an inverted pyramid. In the Second phase it uses the same logic using more full adders. The multipliers, in this paper, are for unsigned operands.

As Wallace multiplier has larger time delay and more complexity than the Dadda multiplier, as explained in [8]and in [9], therefore its modified version is only as good as the Original Dadda multiplier.

III. MODIFIED DADDA REDUCTION APPROACH

A. Modifications of Partial Products

Dadda's Unsigned Integers Multiplier can be modified for signed multiplication in following way.

Multiplier

$A=\{a_n,\ldots\ldots\ldots\ldots\ldots a_6,a_5,a_4,a_3,a_2,a_1,a_0\}$

Multiplicand

$B=\{b_n,\ldots\ldots\ldots\ldots\ldots b_6,b_5,b_4,b_3,b_2,b_1,b_0\}$

The modifications are as follows ;

1st Modification: The Most Significant bit of output can be obtained by XOR operation between MSB of multiplier and multiplicand. If one of the number is positive i.e. >0 and other number is negative i.e. <0 then the product will be negative. If both the numbers are positive i.e. >0 or negative i.e. <0 then the product must be positive. Using only one XOR function reduces the complexity of the multiplier.

MSB (multiplier)	MSB (multiplicand)	MSB(output)
0	0	0
0	1	1
1	0	1
1	1	1

2nd Modification: - The partial products $a_{n-1}b_x$ (where x <n-1) are calculated by $A\overline{B}$ (A. (complement of B)) instead of AB. As a_{n-1} is the signed bit the partial product should be complemented when a is 1.

a_{n-1}	b_x	A□
0	0	0
0	1	0
1	0	1
1	1	0

When sign bit is 1 the function complements as rules of two's complement multiplication, as shown in [10]-[12], tells the partial product should be negated.When the sign bit is 0 the partial product remains as it was, it only complements when sign bit is 1 it gives the complemented product.

Similarly the partial products a_xb_{n-1} (where x <n-1) are calculated by $\overline{A}B$ (B. (complement of A)) instead of AB**.** As b_{n-1} is the signed bit the partial product should be complemented when b is 1.

b_{n-1}	a_x	āB
0	0	0
0	1	0
1	0	1
1	1	0

3rd Modification*:* The partial product is $a_{n-1}b_{n-1}$ is calculated as the OR of a_{n-1} and b_{n-1} instead of AND logic gate. Due to the signed bits, when a=1, b=0 product should be 1 and vice-versa. When both a & b =1 then the product will be 1 as 0 will complemented as 2nd sign bit is also 1.

B. Multiplier Configuration Details

In Two's Complement to get the negation of a number, we complement the number then add 1, so they are not equivalent. So a one must be added to those partial products such that they are negated. Instead of adding 1 to each such bit form 7th column to the 15th

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

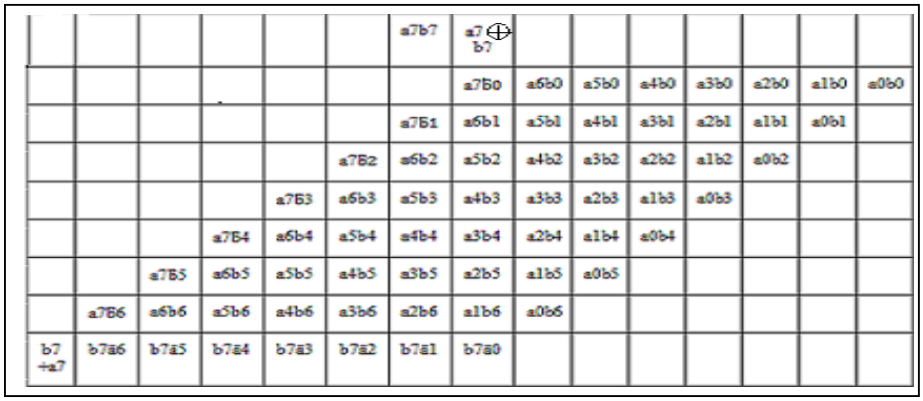


Figure 1 Modified Multiplier Schematic

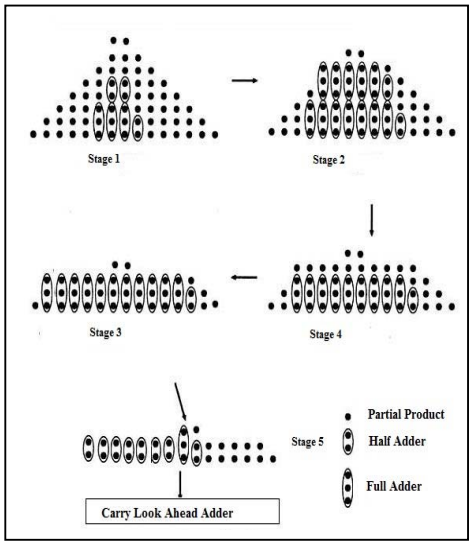


Figure 2 Modified Multiplier Schematic

column one is added in such a way it compensates for all the ones required for negation and -1's in bit columns 7 through 15 (where each of the MSBs are located) are added together.

4th Modification: To 2^{n-1} bit (in 8bit multiplier—7th bit) XOR of $a_{n-1}b_{n-1}$ is added to it i.e. 1 is added to 2n-1 bit if one number is positive and other is negative. It is done in accordance with the rules of signed numbers multiplication. When one unsigned number and a signed number is multiplied, as told the 1 required to be added to the partial products from bit columns 7 to 15, can be simplified to the single 1 that floats "magically" on the 2^{n-1} bit(7th bit in 8bit multiplier).

5th Modification: With 256 bit (2^8bit) in an 8 bit multiplier a_7b_7 is added, which is 1 when both numbers are negative. As 1 has to be added in Two's complement to get the negation of a number, in 8th bit 1 is added when both the numbers are negative to compensate for all the ones in the negation. All the ones that has to be added in the negation can be simplified to 1 floating on the 2^n bit (8th bit) in case of both negative numbers.

By these modifications we reduce the partial products by the Dadda's unsigned multiplier. In the unsigned multiplier only (3, 2) counters and (2, 2) counters are used. In this phase, the Dadda approach uses few stages of full and half adders as carry save adders that are arranged to maximize the reduction at each stage. Full adders take in three bits and output two bits for a net reduction of one bit per full adder. Half adders take in two bits and output two bits. They simply shift the position The N rows of partial products are reduced to two rows by these carry save adders.

The number of (3, 2) and (2, 2) counters required depends on N, the number of bits of operand as

(3, 2) counters = $N^2 - 4.N + 3$

(2, 2) counter = N-1

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

in an unsigned Dadda multiplier [8].

C. Final Stage –CLA adder

Once the matrix has been reduced to height of two, the final stage consists of carry look ahead adder to produce the final result. The size of the CLA adder is determined as follows

CLA Length = 2.N – 2

The CLA adder is faster and has less time delay than equivalent number of full adders which is explained fully in [13].In CLA adder, carry is calculated by generate and propagate and previous carry bits, which is independent of the sum produced, so it works faster than N number of full adders joined together.

IV. RESULTS AND SIMULATION

This section compares the transistor count, power dissipation and time delay for the Dadda signed multiplier and the modified Dadda multiplier. In the Dadda signed multiplier 16 columns (136 partial products) are added through carry save adders while in case of modified Dadda multiplier only 8 columns (64 partial products) are added to get the final result.

Table 2 MULTIPLICATION OF -6(1010)*4 (0100) :- 24(1110100) IN A 4 BIT SIGNED MODIFIED DADDA Multiplier

			0=ab	1=a ⊕ b			
				1	0	0	0
			1	0	0	0	
		0	0	1	0		
	1=(a +b)	0	0	0			
1=(a ⊕b)							
1	1	1	0	1	0	0	0

A. Transistor Count

Due to the low number terms to be added the number of the full adders and half adders required are much less in the modified version than in the Dadda signed multiplier. In Dadda signed multiplier higher compressors (5, 3) and (10, 4) are required than the (3, 2) and (2, 2) compressors. This is done to decrease the complexity, if not it will require large number of full adders for the conventional Dadda multiplier.

Table 3 TRANSISTOR COUNT

	Transistor Count	Full Adders	Half Adders	(5-3) Counters	(10-4) Counters
Modified Multiplier	2236	36	16	Nil	Nil
Conventional Dadda Multiplier	5082	9	8	10	11

Each (5, 3) counter require as much transistor as 2 full adder and 2 half adder and each (10, 4) counter has transistor count equal to 6 Full adder and 3 Half adder.

B Power Dissipation

As transistor count is less, power dissipated in the modified Dadda multiplier is much less than the original Dadda Signed multiplier using the “reduced sign extension logic”. Power dissipated in the modified as well as the conventional Dadda multiplier with varying Vdd as well as frequency.

Table 4 Power vs. Vdd

VDD	1	1.2	1.5	1.8	2	2.5	3
Modified(in µW)	3.6	5.6	9.1	13	1.56	28.7	43.7
Dadda (in mW)	3.7	5.4	8.7	0.12	0.16	0.25	0.37

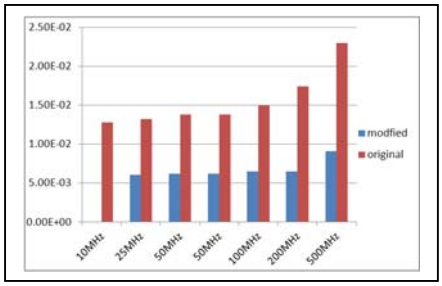


Figure 2 Power Vs Frequency

A. *Time Delay*

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

When the delay is compared, the number of reduction stages as well as the number of compressors used in each reduction stages. In the modified Dadda multiplier has less time delay as it contains less reduction stages and less number of compressors.

Table 5 Time delay vs. VDD

VDD	1	1.2	1.5	1.8	2	2.5	3
Modified (in pico sec)	496	452	206	105	44.4	69.7	65.8
Dadda (in nano sec)	103	98.8	99.9	1.20	1.06	0.85	0.68

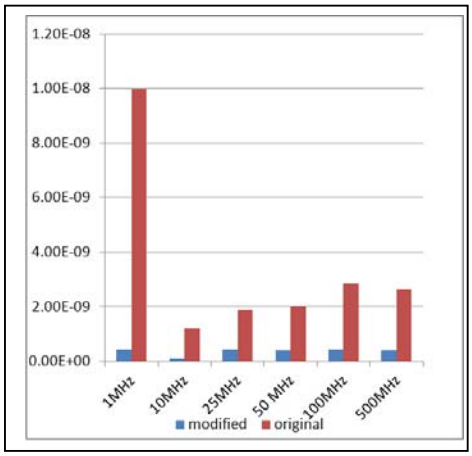


Figure 3 Time delay Vs. Frequency

B. V_{OH} and V_{OL}

The variation of highest voltage and the minimum voltage which represents the logic '1' and logic '0' with variation of frequency in both modified multiplier and the Dadda signed multiplier.

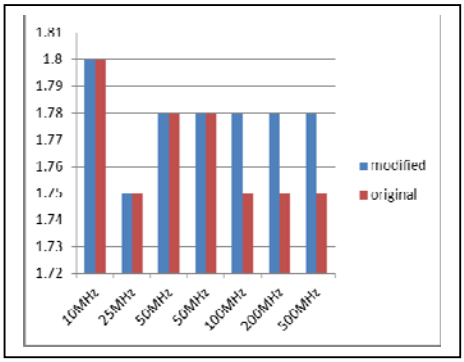


Figure 4 V_{OH} with Frequency

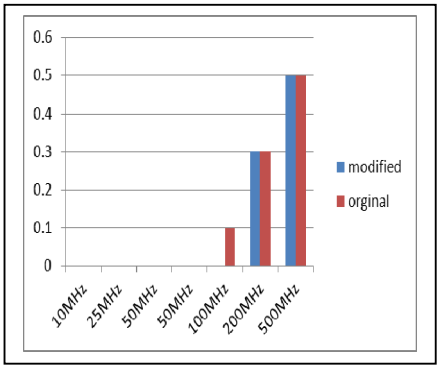


Figure 5 V_{OL} Vs. Frequency

V. CONCLUSIONS

This paper presents a modification to Dadda signed multiplier which is on the basis of reduced sign extension logic. Among the various types of multipliers shown in [14] the modified version can prove to very effective. The modified Dadda multiplier reduces the number of transistors required by at least 50 percent compared to the conventional Dadda signed multiplier. The modified Dadda multiplier has less number of reduction stages and has far less number of power dissipation than the Dadda signed multiplier. It can be implemented using only a (3,2) and (2,2) counters and a final CLA adder.

REFERENCES

[1] Luigi Dadda, On Serial-Input Multipliers for Two's Complement Numbers "Fast multipliers to two's-complement numbers in serial form," in *Proc. IEEE 7th Symp. Comput. Arithmet.*, June 1985, pp. 57-63

[2] L. Dadda, "Some Schemes for Parallel Multipliers," Alta Frequenza, vol. 34, pp. 349-356, 1965.

[3] D.G. Crawley G.A.J. Amaratunga "8 x 8 bit pipelined Dadda multiplier in CMOS"

International Journal on Electronic & Electrical Engineering, IJEEE
ISSN 0974-2042, Volume 11, Issue No 01, June 2010 – August 2010

[4] Asadee "A High-Speed Multiplication Algorithm Using Modified Partial Product Reduction Tree"

[5] Ron S. Waters and Earl E. Swartzlander, Jr. "A Reduced Complexity Wallace Multiplier Reduction"

[6] K.C. Bickerstaff, M.J. Schulte, and E.E. Swartzlander Jr., "Parallel Reduced Area Multipliers," J. VLSI Signal Processing Systems, vol. 9, no. 3, pp. 181-191, Apr. 1995.

[7] C.S. Wallace, "A Suggestion for a Fast Multiplier," IEEE Trans. Electronic Computers, vol. 13, no. 1, pp. 14-17, Feb. 1964.

[8] W.J. Townsend, E.E. Swartzlander Jr., and J.A. Abraham, "A Comparison of Dadda and Wallace Multiplier Delays," Proc. SPIE, Advanced Signal Processing Algorithms, Architectures, and Implementations XIII, pp. 552-560, 2003.

[9] H. Erikssont, P. Larsson-Edeforst, M. Sheerant, M. Sjalandert, D. Johanssont, and M. Schlint , "Multiplier Reduction Tree with Logarithmic Logic Depth and Regular Connectivity"

[10] Qi Wang and Yousef R. Shayan "A Versatile Signed Array Multiplier suitable for VLSI Implementation"

[11] Hennessy and Patterson, "Computer Architecture: A quantitative Approach" Section A.2 (pages A-3 through A-6) and section A.9 (pages A-39 through A-49).

[12] John F. Wakerly "Digital Design Principles & Practices", Prentice Hall, 3rd edition 2000, page 47

[13] A Weinberger and J.L. Smith "A Logic For High Speed Addition"

[14] Gary W. Bewick "Fast Multiplication: Algorithms and Implementation"